

E0. Programar bot inicial v0

En este laboratorio crearéis vuestro primer bot en Python. Heredará de la clase base BotInRedUC3M, que **registra automáticamente los datos** de cada batalla. Ese dataset es la materia prima de los laboratorios siguientes, así que vuestro bot v0 es la pieza que pone en marcha todo el curso.

1. Requisitos

- **GUI de Tank Royale** instalada y funcionando (necesita **Java 11+**).
- **Python 3.10+**. Comprobadlo con `python --version`.
- La librería del bot: `pip install robocode-tank-royale`.
- El fichero BotInRedUC3M.py subido en AulaGlobal.

2. Estructura de vuestro bot

Un bot de Python es una **carpeta** cuyos ficheros comparten su nombre, dentro de una **carpeta raíz** que registraréis en la GUI. Importante: BotInRedUC3M.py debe estar **junto a vuestro bot** para poder importarlo.

```
bots/                                     <- carpeta raíz / root directory (lo añades en la GUI)
├── BotGrupo1/
│   ├── BotInRedUC3M.py                 <- clase base de recogida de datos (en AulaGlobal)
│   ├── BotGrupo1.py                   <- vuestro bot (hereda de BotInRedUC3M)
│   ├── BotGrupo1.json                 <- info del bot
│   ├── BotGrupo1.cmd                  <- arranque (solo Windows)
│   ├── BotGrupo1.sh                   <- arranque (solo macOS/Linux)
│   └── datos_batallas/                <- se crea solo al ejecutar; aquí van los CSV
```

3. Crear el fichero BotGrupo1.json

Describe vuestro bot. name, version y authors son obligatorios:

```
{
  "name": "Bot Grupo 1",
  "version": "0.0",
  "authors": [ "Pareja XX - Nombres y apellidos" ],
  "description": "Bot inicial v0 - Inteligencia en Red",
  "platform": "Python 3.10",
  "programmingLang": "Python"
}
```

4. Crear el fichero BotGrupo1.py

Vuestro bot hereda de BotInRedUC3M y, en `__init__`, le pasa el nombre de vuestro `.json`. Escribid vuestra lógica en los métodos `on_*`, el registro de datos se añade solo. A continuación se añade el esqueleto básico del bot, modificarlo para crear vuestra v0 del bot:

```
from BotInRedUC3M import BotInRedUC3M
from robocode_tank_royale.bot_api.events import ScannedBotEvent

class BotGrupo1(BotInRedUC3M):

    def __init__(self):
        super().__init__("BotGrupo1.json") # vuestro .json
```

```

# Lógica principal: se repite cada turno de la ronda
def run(self):
    while self.running:
        self.set_turn_radar_right(45)    # barrer el radar SIEMPRE
        self.go()                        # cierra el turno

# Os han escaneado a un rival -> aquí va vuestro movimiento
def on_scanned_bot(self, e: ScannedBotEvent):
    self.set_fire(1)

# (Opcional) lógica por turno
def on_tick(self, e):
    pass

def main():
    BotGrupo1().start()

if __name__ == "__main__":
    main()

```

5. Dónde va vuestra lógica

- **Vuestra lógica** va en `run()` y en los métodos `on_*` (`on_scanned_bot`, `on_hit_by_bullet...`), mirad la documentación oficial para añadir algo de inteligencia a vuestro primer bot. El registro de datos se añade solo.
- **Importante:** no llaméis a `super()` dentro de los `on_*` (el registro ya se añade solo; si lo llamáis, se duplicaría la fila).

6. Scripts de arranque

Indican a la GUI cómo lanzar el bot. Para Windows, `BotGrupo1.cmd`:

```
@echo off
python BotGrupo1.py
```

Para macOS y Linux, `BotGrupo1.sh`:

```
#!/bin/sh
python3 BotGrupo1.py
```

En macOS/Linux, dadle permisos: `chmod 755 BotGrupo1.sh`.

7. Ejecutar y comprobar los datos

1. Abrid la **GUI** (arranca el servidor).
2. En **Config** → **Bot Root Directories**, añadid vuestra carpeta raíz (la que contiene `BotGrupo1`).
3. Vuestro bot aparecerá en **Bot Directories**. Seleccionadlo y pulsad **Boot**: se conectará (pasará a **Joined Bots**).
4. Añadidlo a **Selected Bots** junto a un **sample bot** rival y pulsad **Start Battle**.
5. Al terminar, mirad la carpeta `datos_batallas/`: debe haber un CSV `BotGrupo1__<fecha>_<pid>.csv`.

8. Qué contiene el CSV

Cada batalla genera un fichero en `datos_batallas/`. Es una tabla con una fila por evento y siempre las mismas 20 columnas. La columna `tipo` dice qué representa la fila y la columna `id`, a qué bot se refiere. Las columnas que no aplican a ese tipo de fila se quedan vacías, no es un error. Para ordenar cronológicamente usad `ronda` y `turno`.

Tipos de fila (columna `tipo`)

tipo	Qué registra	Cuándo se genera
<code>propio</code>	Estado de vuestro bot: posición, energía, velocidad y orientación.	En cada turno.
<code>rival</code>	Lo que ve el radar de un rival, más la distancia y el ángulo hacia él.	En cada escaneo.
<code>disparo</code>	Bala disparada: origen, dirección, potencia, y a qué rival y a qué distancia se apuntaba.	Al disparar.
<code>impacto_dado</code>	Una bala vuestra impacta a un rival.	Al acertar.
<code>impacto_recibido</code>	Os impacta la bala de otro bot.	Al recibir el impacto.
<code>choque_pared</code>	Chocáis contra la pared.	Al chocar.
<code>choque_bot</code>	Chocáis contra otro bot.	Al chocar.
<code>fin_ronda</code>	Lo que consiguió vuestro bot en esa ronda y si sobrevivió.	Al acabar cada ronda.
<code>fin_juego</code>	Totales de vuestro bot en toda la batalla.	Al acabar la batalla.

Significado de las columnas

Columna	Qué es y dónde aparece
<code>ronda</code>	Número de ronda dentro de la batalla, empezando en 1. En todas las filas.
<code>turno</code>	Turno dentro de la ronda en el que se registró la fila. En todas las filas.
<code>tipo</code>	Qué representa la fila (tabla anterior). En todas las filas.
<code>id</code>	Bot al que se refiere la fila: el vuestro en las filas propias y de eventos, el rival escaneado en <code>rival</code> .
<code>x, y</code>	Posición en la arena, en píxeles, con el origen abajo a la izquierda. En <code>propio</code> , <code>rival</code> , <code>disparo</code> (posición de la bala), <code>choque_pared</code> y <code>choque_bot</code> .
<code>energia</code>	Energía del bot. La vuestra en <code>propio</code> ; la del rival escaneado en <code>rival</code> ; la energía tras el impacto en las filas de impacto.
<code>velocidad</code>	Velocidad en píxeles por turno; negativa si va marcha atrás. En <code>propio</code> , <code>rival</code> y <code>choque_pared</code> .

direccion	Orientación del cuerpo en grados, de 0 a 360. En propio, rival y choque_pared. En disparo es la dirección de la bala.
dist_al_rival	Distancia en píxeles entre vuestro bot y el rival. En rival es la del escaneo. En disparo es la del último escaneo, que puede tener ya unos turnos: es la distancia que conocía el bot al decidir disparar.
bearing_al_rival	Ángulo relativo desde vuestro cuerpo hasta el rival, en grados de -180 a 180. Solo en rival.
id_otro	El otro bot implicado en el evento: quién os disparó, a quién acertasteis, con quién chocasteis. En disparo, el último rival escaneado.
potencia	Potencia de la bala, entre 0.1 y 3. En disparo y en las filas de impacto.
dano	Daño causado por ese impacto concreto.
vivo	1 si vuestro bot terminó la ronda vivo, 0 si murió. Solo en fin_ronda. En ese caso turno es el turno en el que murió.
rank	Puesto en la clasificación acumulada: 1 es el primero.
supervivencia	Puntos por seguir vivo cuando cae otro bot: 50 por cada bot derribado estando vosotros vivos.
dano_balas	Puntos por daño hecho con balas: 1 punto por punto de daño.
dano_embestida	Puntos por daño hecho embistiendo: 2 por punto de daño.
puntuacion	Suma de las columnas anteriores. Es lo que determina el rank.

En fin_ronda las columnas de puntuación son **lo conseguido en esa ronda**, si sumáis todas os debe dar la fila fin_juego.

Las columnas de resultados (rank, supervivencia, puntuacion...) solo aparecen en fin_ronda y fin_juego y se refieren siempre a vuestro bot: **el servidor envía a cada bot únicamente sus propios resultados**. De los rivales tenéis solo lo que ha visto vuestro radar.

9. Problemas frecuentes

- **ModuleNotFoundError: robocode_tank_royale:** falta `pip install robocode-tank-royale` (o no activasteis el entorno virtual).
- **Funciona en el terminal pero desde la GUI sigue dando ModuleNotFoundError: robocode_tank_royale:** ya tenéis la librería instalada, pero la GUI arranca el bot con un entorno distinto al de vuestro terminal y puede usar **otro python3** que no la tiene. Solución: averiguad la ruta del `python3` que sí funciona y ponedla en el script. En el terminal, ejecutad `which python3` (macOS/Linux) o `where python` (Windows) y comprobad que es el correcto con:

```
$(which python3) -c "import robocode_tank_royale"
```

Si no imprime error, es ese. Sustituid entonces `python3` por esa **ruta completa** en `BotGrupo1.sh` (o `python` en `BotGrupo1.cmd`). Por ejemplo, si `which python3` devuelve `/opt/local/bin/python3`:

```
#!/bin/sh  
/opt/local/bin/python3 BotGrupo1.py
```

- **ModuleNotFoundError: BotInRedUC3M:** poned `BotInRedUC3M.py` en la misma carpeta que vuestro `.py`.
- **El tanque no se mueve:** revisad que el bucle usa `while self.running:` y que llamáis a `go()`.

Entrega

Empaquetad la carpeta de vuestro bot en un **.zip** (`.py`, `.json` y `scripts`) y subidlo a **Aula Global**. No incluyáis la carpeta `datos_batallas/`.